Experiment 5

Aim: Deploying a Voting/Ballot Smart Contract

Theory:
Importance of 'require' Statements in Solidity

In Solidity, the require statement is used to enforce conditions before a function continues executing. It acts as a safety check to make sure that inputs, permissions, or other requirements are valid. If the condition inside require evaluates to false, the function stops immediately and all changes made during that transaction are reversed.

This behavior protects the contract from invalid operations and keeps blockchain data consistent.

For example, in a Voting Smart Contract, require can verify:

- Whether the caller has permission to vote (voters[msg.sender].weight > 0).
- Whether the voter has already cast a vote.
- Whether the caller is the chairperson before allowing them to assign voting rights.

By using require, developers can enforce rules, prevent unauthorized actions, and provide helpful error messages that make debugging and interaction easier.

Keywords: mapping, storage, and memory

<u>mapping</u>

A mapping is a key–value data structure in Solidity that connects one type of data to another. It is written using the syntax mapping(keyType => valueType).

Example:

```
mapping(address => Voter) public voters;
```

This associates each Ethereum address with a Voter structure containing its information. Mappings are commonly used for quick data retrieval, such as tracking balances or voting details. However, they do not store length information and cannot be directly looped through.

<u>storage</u>

storage refers to the permanent data area of a smart contract on the blockchain. State variables declared in the contract are stored here by default.

Since storage persists across transactions, it is used for important data such as voter records or contract balances. However, modifying storage consumes more gas because the data is written to the blockchain.

memory

memory is temporary storage used during function execution. Variables stored in memory exist only while the function runs and are removed afterward.

It is typically used for intermediate calculations, function parameters, or temporary data structures. Compared to storage, memory operations are cheaper in terms of gas because the data is not permanently recorded on the blockchain.

Why bytes32 Instead of string?

Earlier versions of the Ballot contract used bytes32 to store proposal names instead of string. This choice was mainly for gas efficiency and performance.

bytes32 is a fixed-size type that stores exactly 32 bytes. Because its size is predictable, storing and comparing values requires less computation and therefore less gas. The drawback is that it restricts text to a limited length.

string, on the other hand, is a dynamic data type that can hold text of varying length. This makes it easier to work with and more readable for users, but it requires additional processing inside the Ethereum Virtual Machine, which increases gas costs.

As a result, developers often choose:

- bytes32 when efficiency and lower gas usage are important.
- string when flexibility and readability are preferred.

Many modern contracts now favor string for better usability, even though it may slightly increase gas consumption.

Code:

```
// SPDX-License-Identifier: GPL-3.0
pragma solidity ^0.8.0;

/**
 * @title Ballot
 * @dev Implements voting process along with vote delegation
 */
contract Ballot {

    struct Voter {
        uint weight;      // weight is accumulated by delegation
        bool voted;       // if true, that person already voted
```

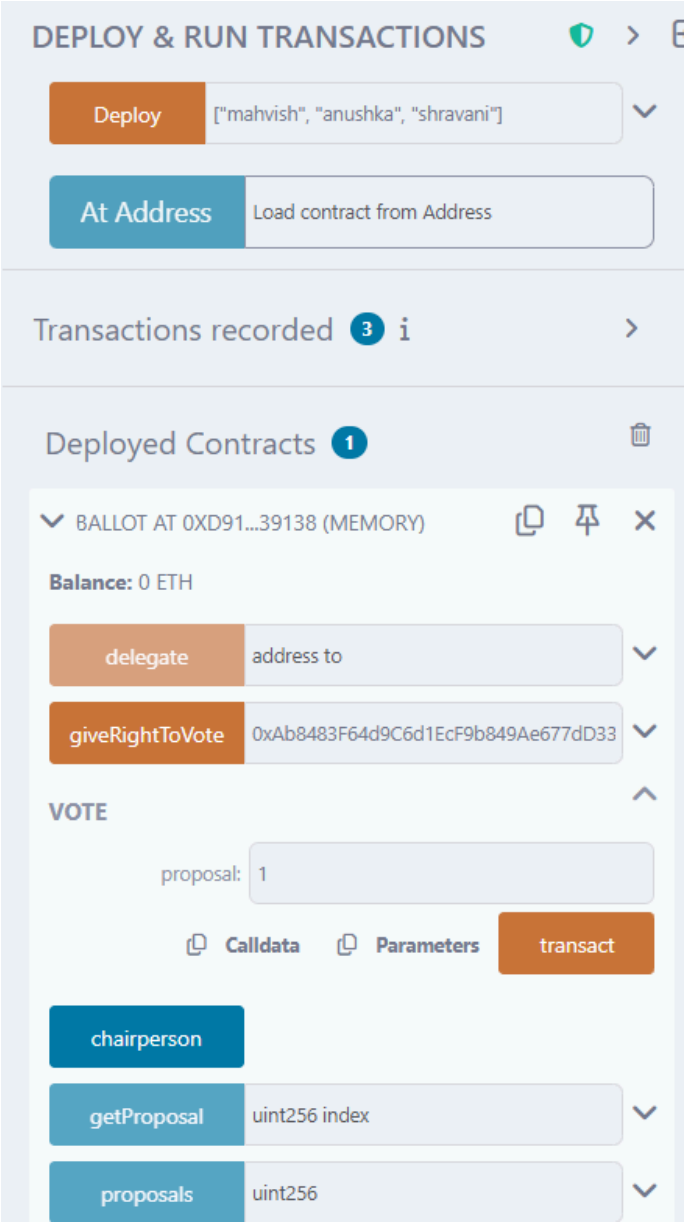
```
    address delegate; // person delegated to
    uint vote;        // index of the voted proposal
}

struct Proposal {
    string name;      // proposal name (changed from bytes32 to string)
    uint voteCount;   // number of accumulated votes
}

address public chairperson;

mapping(address => Voter) public voters;
Proposal[] public proposals;

/**
 * @dev Create a new ballot to choose one of 'proposalNames'.
 * @param proposalNames names of proposals
 */
constructor(string[] memory proposalNames) {
    chairperson = msg.sender;
    voters[chairperson].weight = 1;

    for (uint i = 0; i < proposalNames.length; i++) {
        proposals.push(Proposal({
            name: proposalNames[i],
            voteCount: 0
        }));
    }
}

/**
 * @dev Give 'voter' the right to vote. Only chairperson can call.
 */
function giveRightToVote(address voter) external {
    require(msg.sender == chairperson, "Only chairperson can give right.");
    require(!voters[voter].voted, "Already voted.");
    require(voters[voter].weight == 0, "Already has right.");

    voters[voter].weight = 1;
}

/**
 * @dev Delegate your vote to another voter
 */
function delegate(address to) external {
    Voter storage sender = voters[msg.sender];

    require(sender.weight != 0, "No right to vote");
    require(!sender.voted, "Already voted");
    require(to != msg.sender, "Self-delegation not allowed");

    while (voters[to].delegate != address(0)) {
        to = voters[to].delegate;
        require(to != msg.sender, "Delegation loop detected");
    }
```

```
    Voter storage delegate_ = voters[to];
    require(delegate_.weight >= 1, "Delegate has no right");

    sender.voted = true;
    sender.delegate = to;

    if (delegate_.voted) {
        proposals[delegate_.vote].voteCount += sender.weight;
    } else {
        delegate_.weight += sender.weight;
    }
}

/**
 * @dev Vote for proposal by index
 */
function vote(uint proposal) external {
    require(proposal < proposals.length, "Invalid proposal");

    Voter storage sender = voters[msg.sender];

    require(sender.weight != 0, "No right to vote");
    require(!sender.voted, "Already voted");

    sender.voted = true;
    sender.vote = proposal;

    proposals[proposal].voteCount += sender.weight;
}

/**
 * @dev Compute winning proposal index
 */
function winningProposal() public view returns (uint winningProposal_) {
    uint winningVoteCount = 0;

    for (uint p = 0; p < proposals.length; p++) {
        if (proposals[p].voteCount > winningVoteCount) {
            winningVoteCount = proposals[p].voteCount;
            winningProposal_ = p;
        }
    }
}

/**
 * @dev Get winner name
 */
function winnerName() external view returns (string memory winnerName_) {
    winnerName_ = proposals[winningProposal()].name;
}

/**
 * ✅ EXTRA FUNCTION
 * @dev Get proposal details by index
 */
function getProposal(uint index) external view returns (string memory, uint) {
```

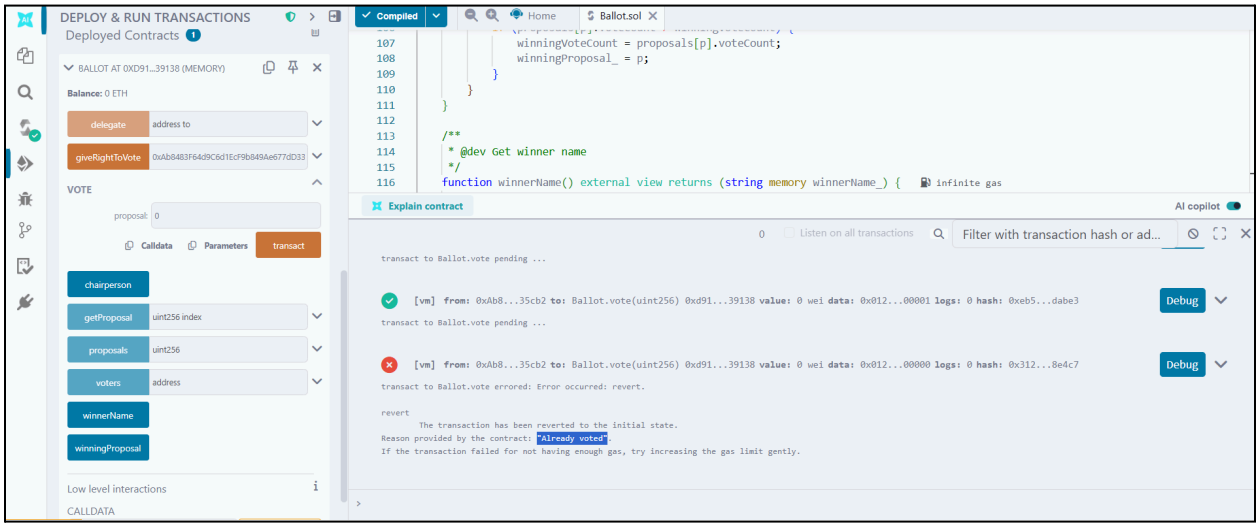
```
        require(index < proposals.length, "Invalid index");
        return (proposals[index].name, proposals[index].voteCount);
    }
}
```

Output:

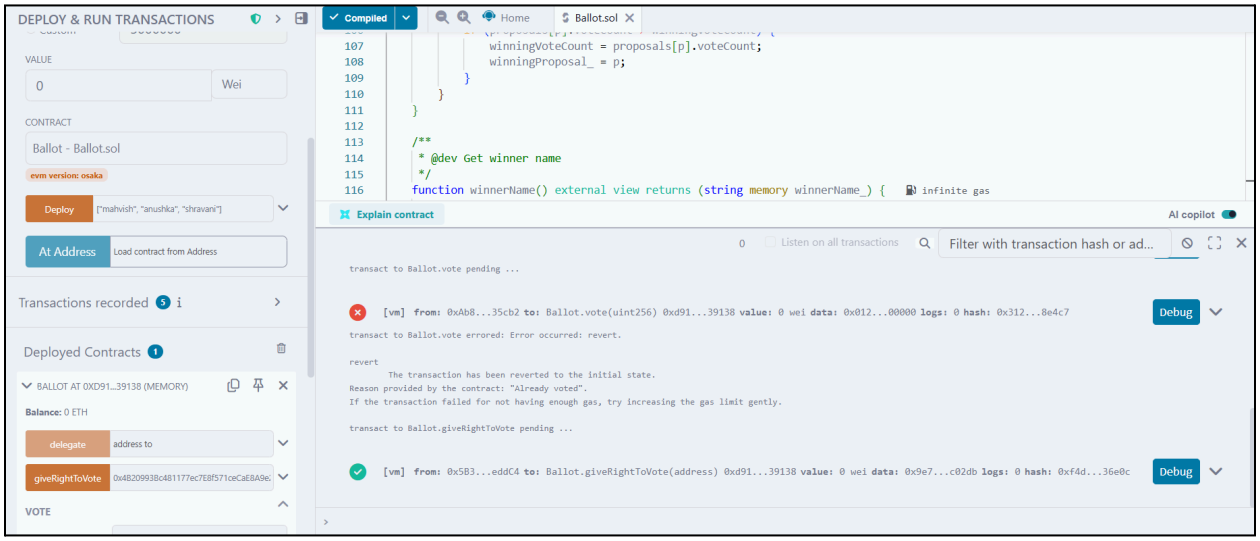
Compiled Ballot.sol Contract



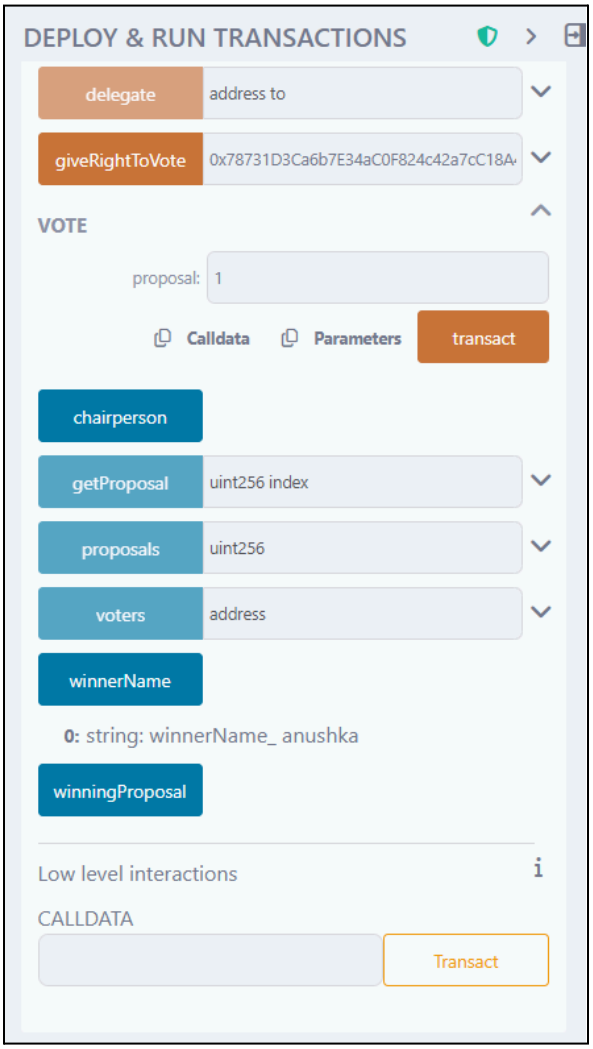
Deploying and running of the contract



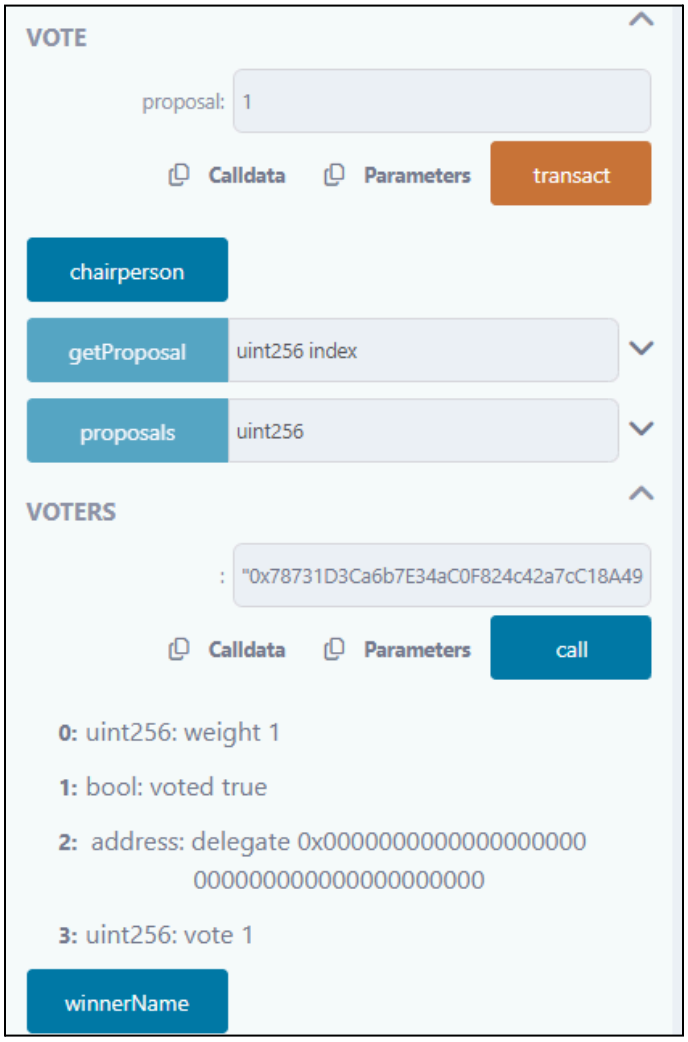
Add 3 candidates



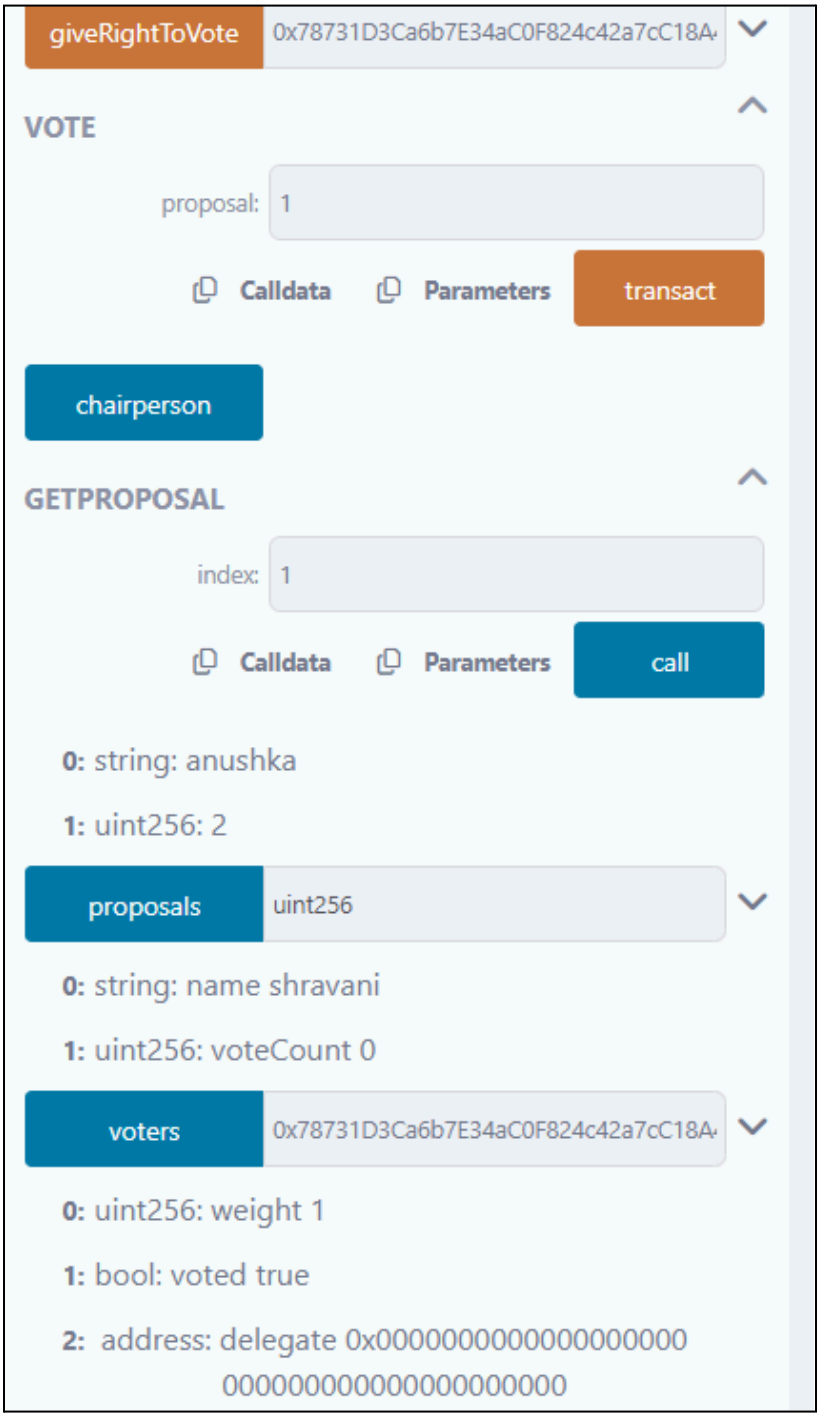
Give right to vote



Cast vote

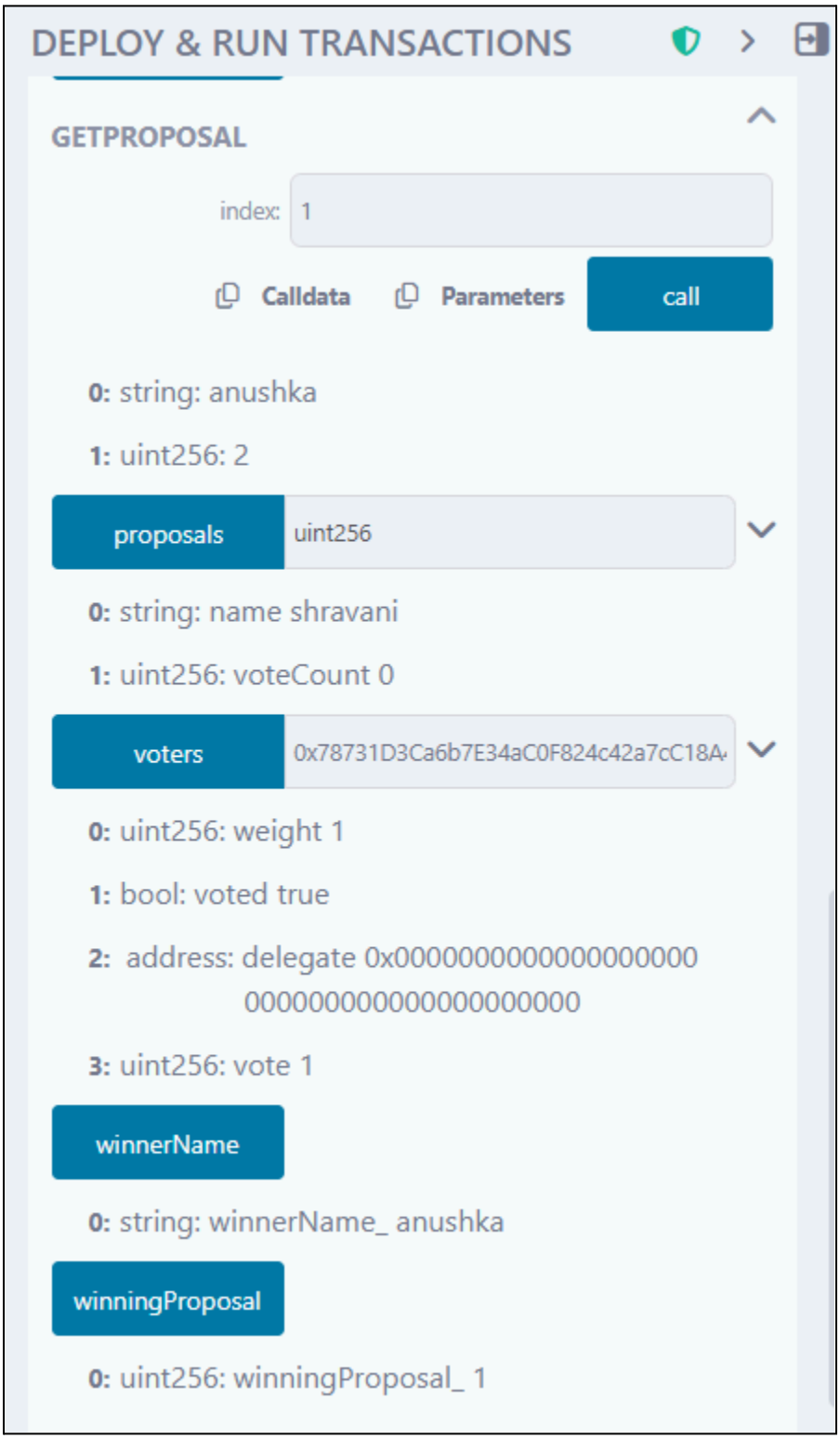


Get candidate info for proposal 2:



Check if a user has already voted

Check winner name:



Conclusion:

In this experiment, a Voting/Ballot smart contract was deployed using Solidity on the Remix IDE. The concepts of require statements, mapping, and data location specifiers like storage and memory were explored to understand their role in ensuring security, efficiency, and correctness in smart contracts. The difference between using bytes32 and string for proposal names was also studied, highlighting the trade-off between gas efficiency and readability. Overall, the experiment provided practical insights into the design and deployment of voting contracts on the blockchain.